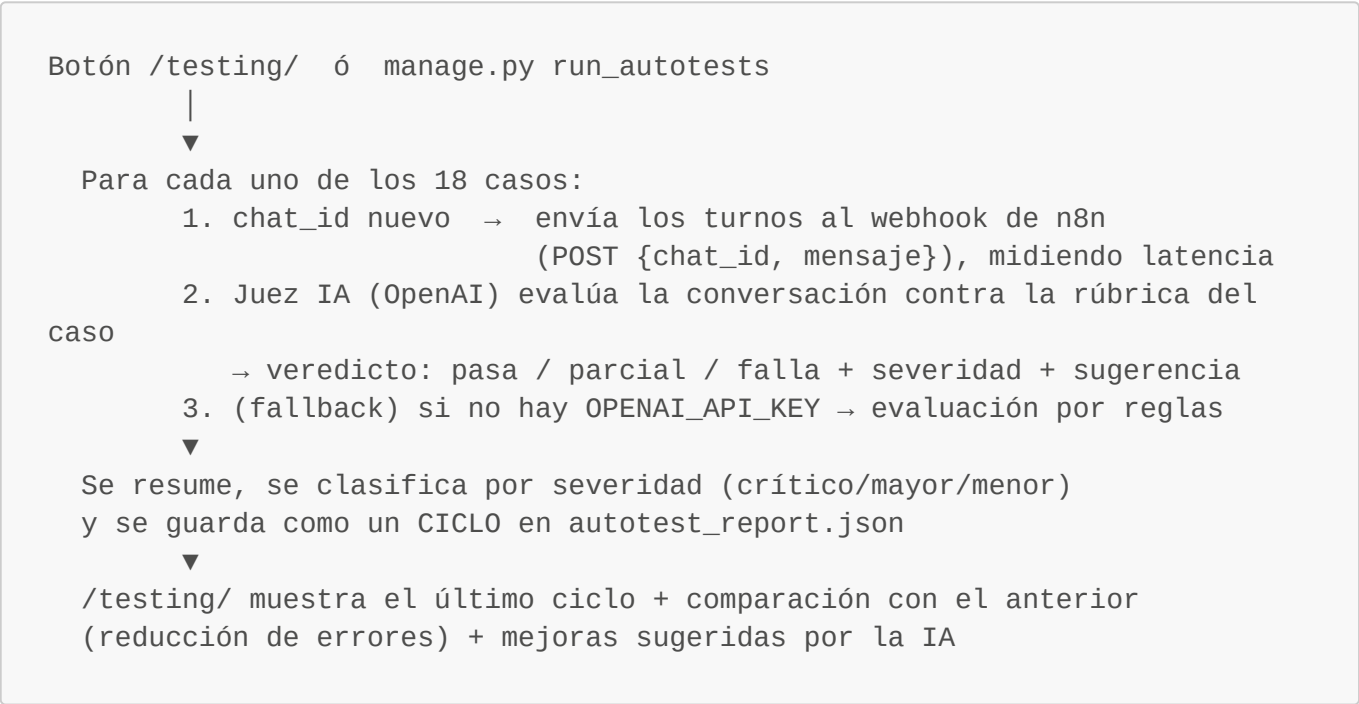


Auto-testing de flujos críticos — Aurorita

Suite de pruebas automatizadas sobre el ecosistema conversacional de Aurora Financiera. Cubre el requisito de **Auto-testing** del reto (Fase 3 y 4): *"mínimo 15 casos sobre los flujos críticos, documentar hallazgos, aplicar las mejoras sugeridas por la IA y demostrar la reducción de errores"*.

- **18 casos** de prueba sobre los **7 flujos conversacionales críticos**.
- Se ejecutan **contra el webhook real de n8n** (no simulado): mismo contrato que usa el navegador.
- Cada caso se **evalúa con un juez IA (OpenAI)** contra una rúbrica; si no hay API key, cae a una evaluación **por reglas** para que la suite siempre corra.
- Los resultados se guardan por **ciclos** y se ven en la web: **/testing/**.

Cómo funciona (flujo de ejecución)



El **ciclo de mejora del reto** se cumple así: se corre el **ciclo 1**, se aplican en n8n las mejoras que sugiere la IA, y se vuelve a ejecutar → el **ciclo 2** muestra en la web la reducción de hallazgos frente al ciclo 1.

Los 7 flujos críticos y los 18 casos

Flujo crítico	Casos	Qué verifican
Onboarding	ON-01, ON-02	Ruteo a vinculación + consentimiento Habeas Data ; si se niega, el flujo se detiene
Consulta de saldo	SA-01, SA-02, SA-03	Autenticación por cédula, pedir consentimiento antes de exponer datos, manejo de cédula inexistente
Crédito	CR-01, CR-02	Precalificación; no prometer aprobación bajo presión

Flujo crítico	Casos	Qué verifican
Mora	MO-01, MO-02, MO-03	Tono empático (mora temprana) / formal (mora dura) / nunca agresivo
Pago	PA-01, PA-02	Ruteo a pago + confirmación clara del pago
Cita	CI-01	Agendamiento usando la ubicación del usuario
Escalamiento	ES-01, ES-02	Derivar a agente humano; bloqueo tras 3 intentos de identidad
(transversales)	SEG-01, AC-01, RU-01	Fuera de alcance (no asesoría financiera), accesibilidad por texto , saludo sin pedir datos

Cada caso tiene: **id**, **flujo**, **dimensión**, **canal**, **severidad** (si falla), la conversación (**turnos**) y una **rúbrica** que describe qué es una respuesta correcta (la usa el juez IA). Se usan **cédulas reales de asociados de prueba** (de **aurora_campaña.csv** en Supabase) para los casos autenticados.

Características / criterios que se verifican

Las pruebas no solo miran "si responde"; validan las reglas del reto:

- **Correctitud funcional** — enruta al flujo correcto y resuelve el trámite.
- **Habeas Data (Ley 1581)** — consentimiento explícito antes de usar datos; si se niega, el proceso se detiene.
- **Tono** — empático en mora temprana, formal en mora dura, **sin lenguaje agresivo ni amenazante**.
- **Seguridad de identidad** — máximo 3 intentos antes de bloqueo.
- **Manejo de errores** — respuestas con gracia, sin trazas técnicas ni cifras inventadas; se detectan fallos de conexión con n8n.
- **Cumplimiento / alcance** — no da asesoría financiera ni promete resultados.
- **Accesibilidad** — siempre existe alternativa por texto.
- **Rendimiento** — mide la latencia por turno y alerta si supera el umbral.

Cada hallazgo se clasifica por **severidad: crítico / mayor / menor**.

Cómo se ejecuta

Desde la web (botón): **/testing/** → botón *"Ejecutar suite de pruebas"* → overlay que muestra el progreso **en vivo, caso a caso** (barra + lista con el veredicto de cada uno, vía Server-Sent Events) → al terminar recarga y muestra el informe completo.

Desde consola:

```
source env/bin/activate
python manage.py run_autotests
```

Ambos añaden un ciclo nuevo al informe.

Cómo fue implementado

Sin framework de tests externo; se apoya en la propia infraestructura de Django y en el proxy a n8n que ya existía.

Archivo	Rol
landing/autotesting.py	Motor: definición de los 18 casos, runner contra n8n, juez IA (OpenAI) + fallback por reglas, resumen/severidad y persistencia por ciclos.
landing/management/commands/run_autotests.py	Comando <code>manage.py run_autotests</code> (ejecuta un ciclo por consola).
landing/views.py	<code>testing_report</code> (informe) y <code>run_autotests_now</code> (ejecuta desde el botón, POST).
landing/urls.py	Rutas <code>/testing/</code> y <code>/testing/run/</code> .
landing/templates/landing/testing.html	Informe ejecutivo + botón + overlay de progreso.
autotest_report.json	Informe acumulado (un objeto por ciclo). Ignorado por git.

Detalles clave de implementación:

- **Reutiliza** `landing/n8n_client.py` para hablar con n8n (mismo contrato `{chat_id, mensaje}` y misma tolerancia de respuesta que el chat real).
- **Conversaciones con memoria:** cada caso usa un `chat_id` fresco (`qa-<caso>-<run>`), así los turnos encadenan estado en n8n.
- **Juez IA:** OpenAI (`gpt-4o` por defecto) con *structured outputs* (`json_schema` estricto) → devuelve siempre `{veredicto, severidad, hallazgo, evidencia, sugerencia, confianza}`. Configurable con `OPENAI_API_KEY` y `AUTOTEST_MODEL` en `.env`.
- **Robustez:** si el juez falla, el caso no tumba la suite (queda "parcial"); si no hay API key, se evalúa por reglas heurísticas (palabras agresivas, consentimiento, escalamiento, errores de conexión).
- **Informe por ciclos:** cada ejecución agrega un ciclo; la web compara el último con el anterior para mostrar la **reducción de errores**.